INSTITUTO FEDERAL DE ALAGOAS – IFAL

CAMPUS MACEIÓ

BACHARELADO EM SISTEMAS DE INFORMAÇÃO

ESTRUTURAS DE DADOS

MINI-PROJETO

ÍNDICE INVERTIDO PARA BUSCA TEXTUAL

Gabriel França

Alicia Barros

Professor: Ricardo Nunes

MACEIÓ – AL

2026

1. O que foi feito

O trabalho consiste em procurar palavras em um texto e mostrar os números das linhas onde elas aparecem. Para isso, usamos um dicionário que guarda cada palavra junto de um conjunto com as linhas correspondentes. Esse dicionário é chamado de índice invertido.

O arquivo indice_invertido.py contém as quatro funções pedidas pelo professor: construir_indice, buscar, buscar_e e buscar_ou. Também criamos um menu simples para fazer as buscas, testes com assert e um programa para comparar os tempos das pesquisas.

A busca simples encontra as linhas de uma palavra. A busca E usa a interseção dos conjuntos, deixando as linhas que têm todas as palavras. A busca OU usa a união, reunindo as linhas que têm pelo menos uma delas. Os resultados são devolvidos em listas ordenadas.

2. Testes do código

Os sete grupos de testes passaram. Conferimos os exemplos do enunciado, busca com mais de duas palavras, palavra inexistente, texto vazio, linhas em branco, pontuação, letras maiúsculas e minúsculas, acentos e palavras repetidas. Também conferimos que fazer uma busca não muda o índice.

Exemplo do professor: a palavra "python" aparece nas linhas [1, 3]. A busca E por "python" e "dados" devolve [1]; a busca OU devolve [1, 2, 3].

3. Comparação de desempenho

Para comparar, usamos 3.600 linhas formadas pela repetição de 12 frases sobre programação. Fizemos as mesmas consultas usando varredura (percorrer o texto de novo) e usando o índice. Medimos com time.perf_counter(), sem contar leitura de arquivos ou impressão, e repetimos cada medição três vezes. As tabelas mostram as medianas obtidas após a revisão do código.

Experimento 1 – 1.000 consultas

Medição	Tempo (ms)
Construção do índice	15,37
1.000 buscas por varredura	11870,62
1.000 buscas pelo índice	14,21

O índice levou um tempo para ser construído, mas as buscas feitas depois ficaram muito mais rápidas.

Experimento 2 – Quando o índice compensa

Consultas	Varredura (ms)	Construção + buscas (ms)
1	11,63	14,87
10	118,09	15,24
100	1173,09	19,54
1000	11870,62	30,09

Para conferir melhor o ponto de mudança, também medimos 2 consultas: varredura = 22,76 ms e índice com construção = 14,24 ms.

4. Respostas às questões da análise

1. Qual estratégia foi mais rápida? Em quais situações?

Na medição com uma busca, a varredura foi mais rápida. Nas quantidades maiores, o índice foi melhor, pois já tinha as palavras organizadas.

2. A partir de quantas consultas o índice passou a valer a pena?

Nesse teste, por volta de 2 consultas. Isso pode mudar em outro computador ou com outro texto.

3. Por que usar conjuntos em vez de listas?

Porque eles evitam linhas repetidas e facilitam as operações E (interseção) e OU (união).

4. Qual o custo esperado de buscar uma chave em um dicionário?

Em média, O(1), porque o dicionário usa uma tabela hash. Em casos desfavoráveis, a busca pode ser mais lenta.

5. Qual o custo de construir o índice e fazer união/interseção?

A construção percorre o texto e faz inserções com custo médio constante por palavra, ficando linear no tamanho da entrada. Em conjuntos, a união de A e B custa em média O(|A| + |B|), e a interseção custa em média O(min(|A|, |B|)).

6. Qual a desvantagem de memória?

O índice ocupa espaço extra para guardar as palavras e os números das linhas. A busca por varredura não precisa guardar esse índice.

5. Conclusão

O índice e as três buscas funcionaram nos testes feitos. O experimento também mostrou por que vale a pena construir um índice quando precisamos pesquisar muitas vezes em um mesmo texto. O projeto ajudou a praticar dicionários, conjuntos e funções em Python.